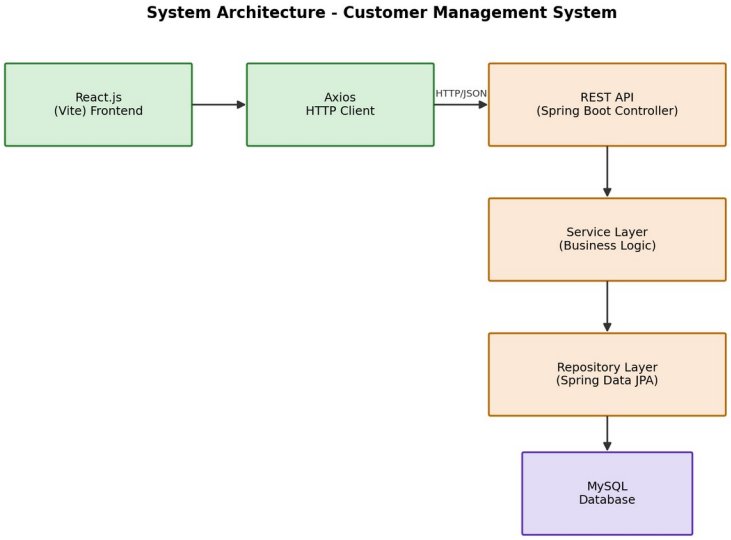
**A PROJECT REPORT ON**

# CUSTOMER MANAGEMENT SYSTEM

*A Full Stack Web Application using Spring Boot, React.js and MySQL*




Submitted in partial fulfillment of the requirements
for the degree / course of Full Stack Java Development

**Submitted by:**

[Student Name]
[Roll Number / Enrollment ID]

**Under the Guidance of**

[Guide / Mentor Name]

**[Institution / College Name]**

Department of Computer Science / Information Technology

2026

# CERTIFICATE

This is to certify that the project report entitled “Customer Management System” is a bonafide work carried out by [Student Name], submitted in partial fulfillment of the requirements for the completion of the Full Stack Java Development course/program.

The project has been carried out under my/our supervision and guidance. To the best of my/our knowledge, the work presented in this report is original and has not been submitted elsewhere for the award of any other degree or diploma.

The Customer Management System demonstrates the practical application of Spring Boot, React.js, and MySQL technologies to build a complete CRUD-based full stack web application.

| | |
|---|---|
| _____ | _____ |
| Project Guide / Mentor | Head of Department |
| [Guide Name] | [HOD Name] |

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me throughout the development of this project, “Customer Management System”.

I am extremely thankful to my project guide/mentor for their invaluable guidance, continuous encouragement, and constructive feedback at every stage of this project. Their expertise in full stack development greatly helped me understand and implement the concepts of Spring Boot, React.js, and REST API design.

I would also like to thank the faculty members of the Department of Computer Science / Information Technology for providing the necessary resources, infrastructure, and a conducive learning environment that made this project possible.

Finally, I extend my heartfelt thanks to my family and friends for their constant motivation and support throughout this journey.

[Student Name]

# ABSTRACT

The Customer Management System is a full stack web application designed to help organizations efficiently manage customer records, including customer name, email, phone number, and city. The system provides a simple yet powerful interface for performing Create, Read, Update, and Delete (CRUD) operations on customer data through a RESTful API.

The backend of the application is built using Java, Spring Boot, Spring Data JPA, and MySQL, following a clean layered architecture consisting of Controller, Service, and Repository layers. It exposes REST APIs secured with input validation and centralized exception handling, ensuring reliable and consistent responses to client requests.

The frontend is developed using React.js with Vite as the build tool, React Router DOM for client-side routing, Axios for HTTP communication, and Bootstrap 5 for a responsive, mobile-friendly user interface. The application allows users to view a searchable list of customers, add new customers, edit existing records, view detailed customer information, and delete customers with confirmation prompts.

This project demonstrates the end-to-end development lifecycle of a modern full stack application — from database design and REST API development to building a polished, responsive single-page application — and serves as a strong foundation for building more advanced enterprise-grade customer relationship management (CRM) solutions.

# TABLE OF CONTENTS

# 1. Introduction

In today's digital era, organizations across every industry rely on efficient systems to manage growing volumes of customer information. Manually maintaining customer records using spreadsheets or paper-based registers is time-consuming, error-prone, and difficult to scale. The Customer Management System addresses this problem by providing a centralized, web-based platform to store, retrieve, update, and delete customer data quickly and reliably.

This project is built using a modern full stack technology stack: Spring Boot for the backend REST API, MySQL for persistent storage, and React.js (with Vite) for a fast, responsive frontend. The system follows industry-standard architectural patterns, including a layered backend architecture, DTO-based data transfer, centralized exception handling, and RESTful API design principles.

# 2. Objectives

- To design and develop a full stack web application that manages customer records efficiently.
- To implement complete CRUD (Create, Read, Update, Delete) functionality using REST APIs.
- To build a normalized relational database schema for storing customer information.
- To apply a clean, layered backend architecture (Controller → Service → Repository).
- To implement robust input validation and centralized exception handling on the backend.
- To develop a responsive, user-friendly frontend using React.js and Bootstrap 5.
- To enable search functionality so users can quickly locate customer records by name.
- To ensure smooth communication between frontend and backend using Axios and properly configured CORS.

# 3. Scope

The scope of this project is limited to managing core customer details — name, email, phone number, and city — through a web-based interface. It covers the complete lifecycle of a customer record: creation, viewing (both list and detail views), searching, updating, and deletion.

While the current version focuses on single-entity (Customer) management, the architecture is designed to be extensible. Future iterations could expand the scope to include order management, customer communication history, authentication and role-based access control, and integration with third-party CRM or ERP systems, as discussed in the Future Enhancements section of this report.

# 4. Software Requirements

| Component | Technology / Tool | Version |
|---|---|---|
| Programming Language | Java | 17 or 21 |
| Backend Framework | Spring Boot | 3.3.x |
| Build Tool (Backend) | Maven | 3.8+ |
| ORM Framework | Spring Data JPA (Hibernate) | Latest |
| Database | MySQL | 8.0+ |
| Frontend Library | React.js | 18.x |

| Component | Technology / Tool | Version |
|---|---|---|
| Frontend Build Tool | Vite | 5.x |
| Routing | React Router DOM | 6.x |
| HTTP Client | Axios | 1.x |
| CSS Framework | Bootstrap | 5.x |
| IDE | Visual Studio Code | Latest |
| Operating System | Windows / Linux / macOS | Any |

## 5. Hardware Requirements

| Component | Minimum Requirement |
|---|---|
| Processor | Intel Core i3 (or equivalent) and above |
| RAM | 4 GB minimum (8 GB recommended) |
| Storage | 10 GB free disk space |
| Display | 1024 x 768 resolution or higher |
| Internet Connection | Required for downloading dependencies |

# 6. System Architecture

The Customer Management System follows a three-tier architecture consisting of the presentation layer (React frontend), the application layer (Spring Boot REST API with Controller, Service, and Repository layers), and the data layer (MySQL database). This separation of concerns improves maintainability, testability, and scalability of the application.

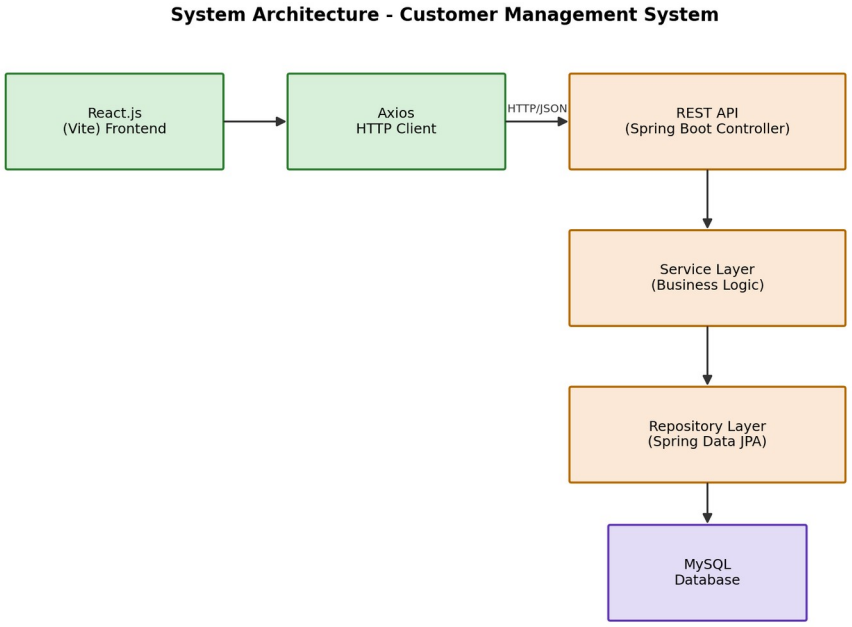



*Fig 6.1: System Architecture Diagram*

# 7. ER Diagram

The Entity-Relationship diagram below represents the single core entity used in this system — the Customer entity — along with its attributes and the primary key constraint.

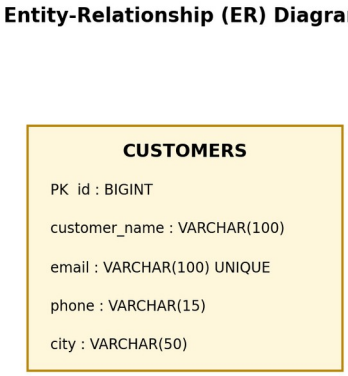



*Fig 7.1: Entity-Relationship Diagram*

# 8. Database Design

The application uses a single, well-normalized table named customers within the customer_db database to store all customer records. The schema is summarized below.

| Column Name | Data Type | Constraints | Description |
|---|---|---|---|
| id | BIGINT | Primary Key, Auto Increment | Unique identifier for each customer |
| customer_name | VARCHAR(100) | NOT NULL | Full name of the customer |
| email | VARCHAR(100) | NOT NULL, UNIQUE | Customer's email address |
| phone | VARCHAR(15) | NOT NULL | Customer's contact number |
| city | VARCHAR(50) | NOT NULL | City where the customer resides |

**Database Name: customer_db  Table Name: customers**

# 9. Use Case Diagram

The use case diagram illustrates the interactions between the Admin User (the primary actor) and the core functionalities offered by the Customer Management System.

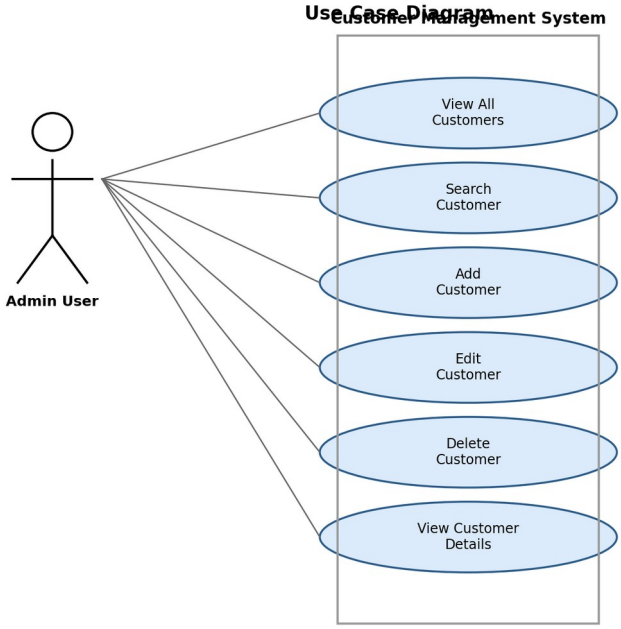



*Fig 9.1: Use Case Diagram*

# 10. Activity Diagram

The activity diagram below depicts the general workflow followed when a user interacts with the system to perform a CRUD operation on customer data, from launching the application to receiving a success or error message.

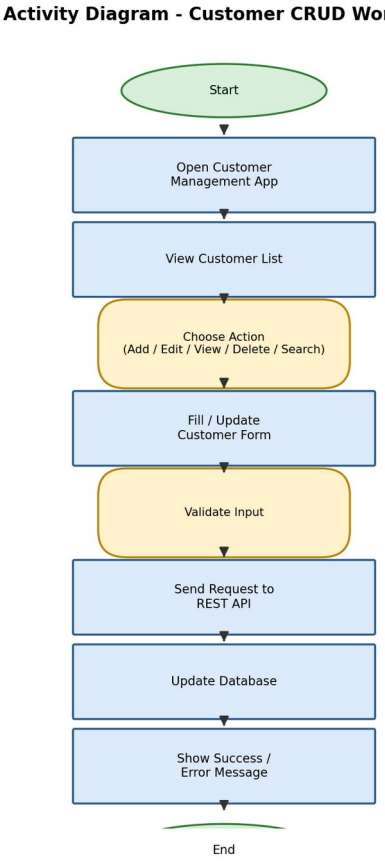



*Fig 10.1: Activity Diagram*

# 11. Class Diagram

The class diagram represents the key backend classes — the Customer entity, CustomerDTO, CustomerRepository, CustomerService (and its implementation), and CustomerController — and the relationships between them, reflecting the layered architecture of the application.

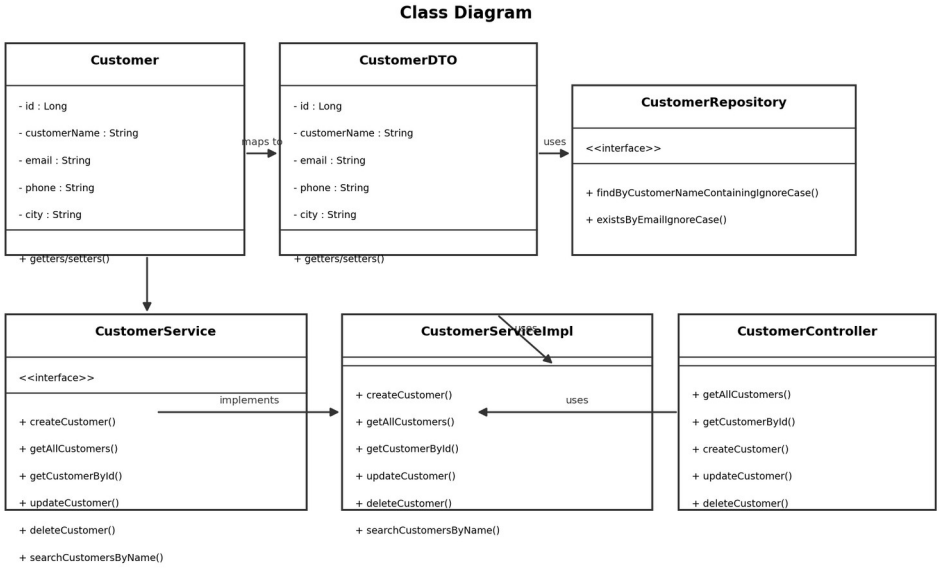



*Fig 11.1: Class Diagram*

# 12. Sequence Diagram

The sequence diagram illustrates the flow of a typical request (e.g. fetching or saving a customer) as it passes from the React frontend through the Controller, Service, and Repository layers, down to the MySQL database and back.

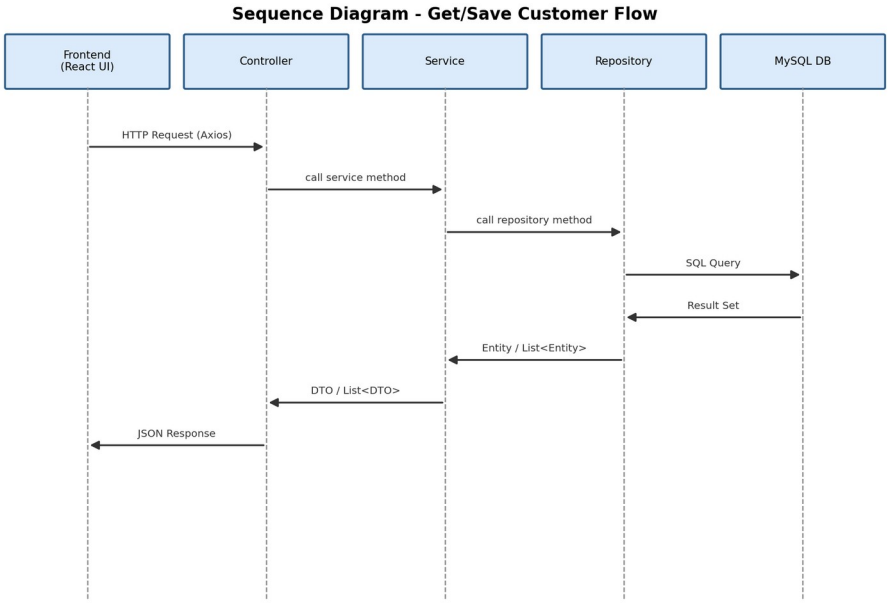


*Fig 12.1: Sequence Diagram*

# 13. Project Modules

| Module | Description |
|---|---|
| Customer List Module | Displays all customer records in a responsive, searchable table with options to view, edit, or delete each record. |
| Add Customer Module | Provides a validated form to capture and submit new customer details to the backend. |
| Edit Customer Module | Loads existing customer data into a form and allows the user to update and save changes. |
| View Customer Module | Displays the complete details of a single customer in a read-only card view. |
| Search Module | Allows users to filter the customer list by searching for a name (case-insensitive, partial match). |
| Delete Module | Removes a customer record after a confirmation prompt, preventing accidental deletions. |
| Validation Module | Performs both client-side (React) and server-side (Spring Validation) validation on all customer fields. |
| Exception Handling Module | Centralized handler that returns consistent, meaningful error responses for not-found, duplicate, and validation errors. |

# 14. API Documentation

The following REST endpoints are exposed by the Spring Boot backend under the base path /api/customers:

| HTTP Method | Endpoint | Description |
|---|---|---|
| GET | /api/customers | Retrieve all customers |
| GET | /api/customers?name={term} | Search customers by name |
| GET | /api/customers/{id} | Retrieve a customer by ID |
| POST | /api/customers | Create a new customer |
| PUT | /api/customers/{id} | Update an existing customer |
| DELETE | /api/customers/{id} | Delete a customer by ID |

**Sample Request Body (POST / PUT):**

```
{   "customerName": "Rahul Sharma",   "email": "rahul.sharma@example.com",
"phone": "9876543210",   "city": "Hyderabad" }
```

**HTTP Status Codes Used:**

- 200 OK — Request processed successfully (GET, PUT, DELETE)
- 201 Created — Customer created successfully (POST)
- 400 Bad Request — Validation errors in the request body
- 404 Not Found — Customer not found for the given ID
- 409 Conflict — Duplicate email address
- 500 Internal Server Error — Unexpected server-side error

# 15. Screenshots

The following placeholders indicate where application screenshots should be inserted once the project is deployed and running locally.

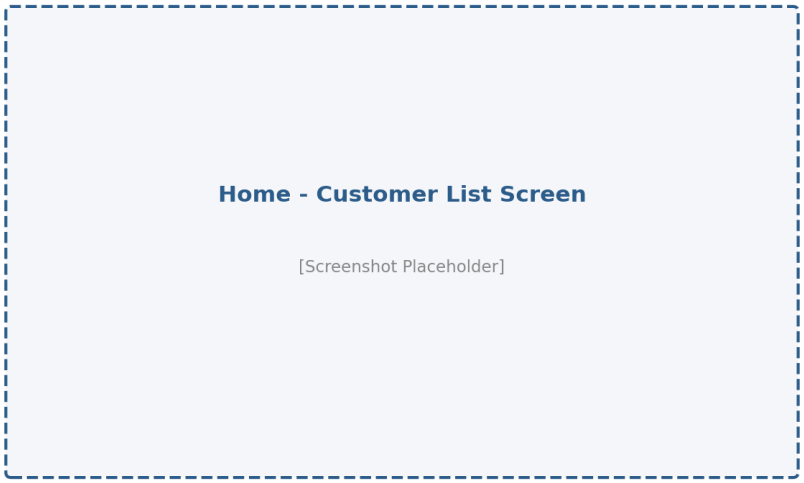



*Fig 15.1: Home / Customer List Screen*

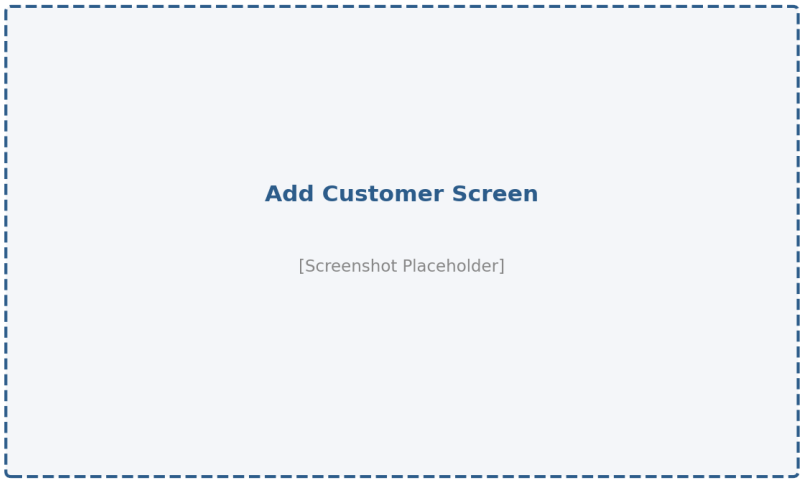



*Fig 15.2: Add Customer Screen*

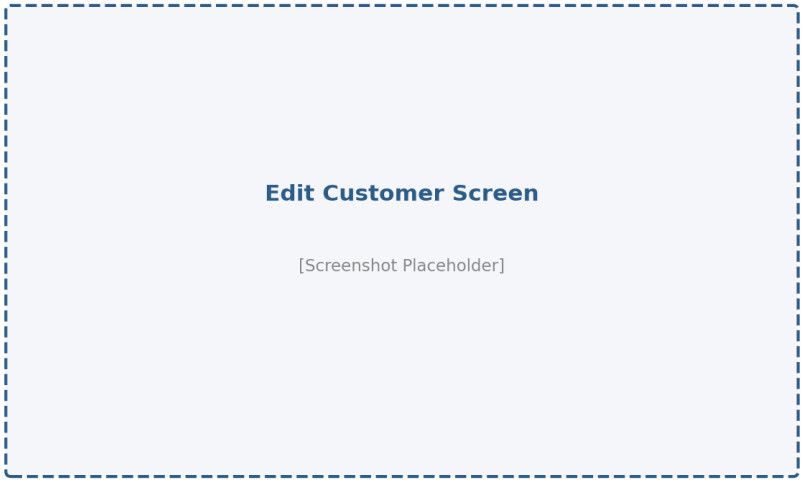



*Fig 15.3: Edit Customer Screen*

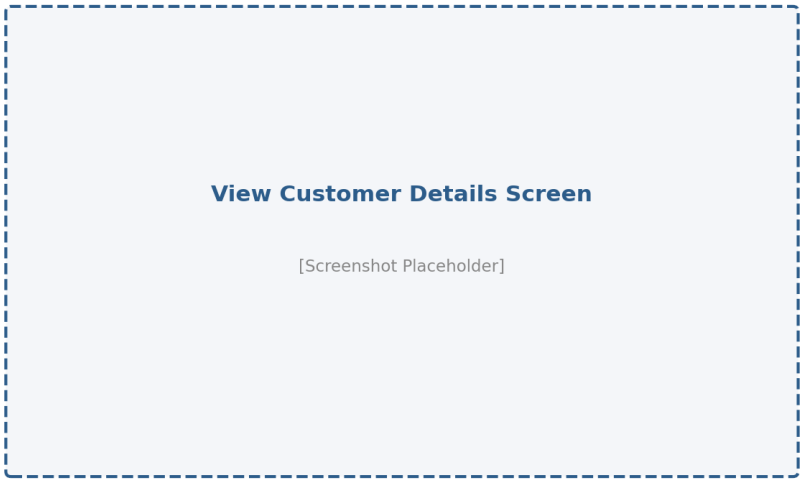



*Fig 15.4: View Customer Details Screen*

# 16. Testing

Testing was carried out at multiple levels to ensure the reliability and correctness of the Customer Management System, covering both backend and frontend functionality.

### 16.1 Unit Testing

Spring Boot's testing framework (spring-boot-starter-test) was used to verify that the application context loads successfully and that core service methods behave as expected.

### 16.2 Manual / Functional Testing

| Test Case | Input | Expected Result | Status |
|---|---|---|---|
| Add customer with valid data | Valid name, email, phone, city | Customer created, success message shown | Pass |
| Add customer with invalid email | Malformed email address | Validation error displayed | Pass |
| Add customer with duplicate email | Existing email address | 409 Conflict error returned | Pass |
| View customer list | N/A | All customers displayed in table | Pass |
| Search customer by name | Partial name match | Filtered list returned | Pass |
| Edit existing customer | Updated field values | Customer updated successfully | Pass |
| Delete customer | Valid customer ID | Customer removed after confirmation | Pass |
| Fetch non-existent customer | Invalid ID | 404 Not Found error returned | Pass |

### 16.3 API Testing

All REST endpoints were tested using tools such as Postman to verify correct HTTP status codes, response payloads, and error handling for both valid and invalid inputs.

# 17. Conclusion

The Customer Management System successfully demonstrates the design and implementation of a complete full stack web application using Spring Boot, React.js, and MySQL. The project showcases a clean, layered backend architecture, a RESTful API design following standard conventions, robust validation and exception handling, and a responsive, user-friendly frontend interface.

Through this project, key full stack development concepts — including database design, REST API development, cross-origin resource sharing, client-side routing, and form validation — were applied in a practical, real-world scenario. The resulting application provides a solid foundation that can be extended into a more comprehensive customer relationship management (CRM) solution.

# 18. Future Enhancements

- Implement user authentication and role-based authorization (e.g. Admin, Staff) using Spring Security and JWT.
- Add pagination and sorting to the customer list for handling large datasets efficiently.
- Introduce advanced filtering (by city, date added, etc.) in addition to name-based search.
- Add customer profile pictures and document/file attachment support.
- Integrate email/SMS notifications for customer-related events.
- Build an analytics dashboard showing customer distribution by city and growth trends over time.
- Containerize the application using Docker and set up CI/CD pipelines for automated deployment.
- Migrate the frontend to TypeScript for improved type safety and maintainability.

# 19. References


- Spring Boot Official Documentation – https://spring.io/projects/spring-boot
- Spring Data JPA Reference Documentation – https://spring.io/projects/spring-data-jpa
- React.js Official Documentation – https://react.dev
- Vite Official Documentation – https://vitejs.dev
- React Router DOM Documentation – https://reactrouter.com
- Axios Documentation – https://axios-http.com
- Bootstrap 5 Documentation – https://getbootstrap.com
- MySQL 8.0 Reference Manual – https://dev.mysql.com/doc
- Baeldung – Spring Boot Tutorials – https://www.baeldung.com